

Rapport TD 2 – Cryptographie

Implémentation et analyse de TEA

Réseau de Feistel de TEA

1.1. Expression de $l_1 || r_1$ et $l_2 || r_2$

D'après le diagramme d'une itération du réseau de Feistel, la branche de gauche prend directement la valeur de la branche de droite précédente, et la branche de droite subit l'opération d'addition modulo 2^{32} avec la fonction F .

Premier tour :

$$l_1 = r_0 \quad r_1 = l_0 + F_0(r_0, k)$$

Second tour :

$$l_2 = r_1 \quad r_2 = l_1 + F_1(r_1, k) = r_0 + F_1(r_1, k)$$

1.2. Inversion de la fonction F pour le déchiffrement

Il n'est **pas nécessaire** d'inverser la fonction F pour déchiffrer un bloc. C'est d'ailleurs tout l'avantage de la structure de Feistel : l'algorithme de chiffrement et de déchiffrement utilisent exactement la même fonction non inversible. Il suffit de remplacer l'addition par une soustraction pour remonter les étapes.

1.3. Expression de $l_0 || r_0$

En appliquant la logique inverse :

$$r_0 = l_1 \quad l_0 = r_1 - F_0(r_0, k) = r_1 - F_0(l_1, k)$$

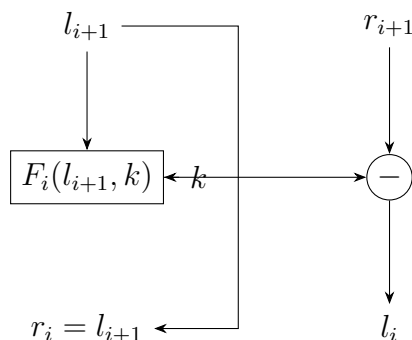
De même, à partir du tour 2 :

$$r_1 = l_2 \quad l_1 = r_2 - F_1(r_1, k) = r_2 - F_1(l_2, k)$$

1.4. Diagramme d'une itération pour le déchiffrement

La logique du diagramme inverse (pour retrouver l'état i à partir de l'état $i + 1$) est la suivante :

- L'entrée de gauche l_{i+1} devient directement la sortie de droite r_i .
- L'entrée de gauche l_{i+1} passe dans la fonction F_i avec la clé k .
- Le résultat de F_i est soustrait (modulo 2^{32}) à l'entrée de droite r_{i+1} .
- Le résultat de cette soustraction devient la sortie de gauche l_i .



Modes de chiffrement

2.1. Mode ECB (*Electronic CodeBook*)

a. Déchiffrement d'un message ECB

Pour déchiffrer un message avec ce mode, il suffit d'appliquer la fonction de déchiffrement par bloc (`feistel_dec`) indépendamment sur chaque bloc de texte chiffré c_i avec la clé k :

$$v_i = \text{Decrypt}(c_i, k)$$

2.2. Mode CBC (*Cipher Block Chaining*)

a. Déchiffrement d'un message CBC

Pour déchiffrer un bloc c_i , on le fait passer dans la fonction de déchiffrement de TEA avec la clé k . Ensuite, on applique un XOR ($\oplus$) avec le bloc chiffré précédent c_{i-1} (ou le vecteur iv pour le premier bloc) :

$$v_i = \text{Decrypt}(c_i, k) \oplus c_{i-1}$$

b. Rôle du vecteur d'initialisation et du chaînage

Le vecteur d'initialisation iv introduit de l'aléatoire : chiffrer deux fois le même message avec la même clé mais un iv différent produit un chiffré totalement différent. Le chaînage lie les blocs entre eux, empêchant un attaquant de repérer des motifs redondants (contrairement à l'ECB) ou de réordonner les blocs de manière malveillante.

c. Parallélisation du CBC vs ECB

- **Mode ECB** : chiffrement *et* déchiffrement sont totalement parallélisables (blocs indépendants).
- **Mode CBC** : le chiffrement n'est *pas* parallélisable (le bloc i dépend du résultat du bloc $i - 1$). En revanche, le déchiffrement est parallélisable car tous les blocs chiffrés c_i sont déjà disponibles.

2.3. Mode OFB (*Output FeedBack*)

a. Déchiffrement d'un message OFB

Le déchiffrement est identique au chiffrement : on génère le même flot en chiffrant successivement le iv , puis on applique un XOR entre ce flot et les blocs chiffrés c_i pour retrouver le clair v_i .

b. Chiffrement par flot

Ce mode transforme un chiffrement par blocs en chiffrement par flot car le message clair ne passe *jamais* dans l'algorithme de chiffrement. TEA est uniquement utilisé comme générateur de nombres pseudo-aléatoires (PRNG) pour créer un masque jetable (le flot), combiné au message par un simple XOR.

c. Parallélisation du OFB

La génération du flot n'est *pas* parallélisable (chaque étape dépend de la précédente). En revanche, une fois le flot généré, l'opération XOR avec le message est totalement parallélisable.

d. Pré-calcul

Dans le mode OFB, la génération du flot ne dépend que de k et de iv – elle est totalement indépendante du message clair v . On peut donc **pré-calculer** le flot à l'avance. Ceci est impossible en ECB (l'entrée de l'algorithme est le clair) et en CBC (l'entrée dépend du clair et du chiffré précédent).

e. Vulnérabilité de la réutilisation du IV

Si l'on utilise le même iv (et la même clé) pour chiffrer deux messages différents v et v' , le même flot S est généré :

$$c = v \oplus S, \quad c' = v' \oplus S$$

Un attaquant peut calculer $c \oplus c' = v \oplus v'$; le flot s'annule. Il obtient alors le XOR des deux textes clairs, ce qui permet des attaques statistiques pour retrouver les messages originaux (*Two-time pad attack*).

f. Nécessité d'une fonction `ofb_decrypt` ?

Non. L'opération de chiffrement et de déchiffrement étant strictement identique (XOR avec le flot pré-calculé), `ofb_encrypt` est une fonction *involution* – une fonction `ofb_decrypt` séparée est donc inutile.

Hachage avec la structure de Merkle-Damgård

a. Taille des empreintes produites

TEA traite des blocs de 64 bits. La fonction de compression de Davies-Meyer met à jour le vecteur d'état iv en utilisant la sortie du chiffrement. L'empreinte finale h produite a donc la taille d'un bloc TEA, soit **64 bits**.

b. Taille du vecteur d'initialisation

Le vecteur d'initialisation iv agit comme le texte clair passant dans l'algorithme. Il doit donc avoir la taille d'un bloc TEA, c'est-à-dire **64 bits** (deux mots de 32 bits).

Exploitation de la faille de TEA

a. Vulnérabilité aux collisions de la fonction de hachage

Dans notre construction basée sur Davies-Meyer, le bloc de message M est utilisé comme clé pour TEA :

$$f(iv, M) = \text{encrypt}(M, iv) + iv$$

TEA possède des **clés équivalentes** : inverser le bit de poids fort des deux premiers mots de 32 bits de la clé ne modifie pas le chiffré résultant. Par conséquent, si l'on crée M' en inversant ce bit sur les deux premiers blocs de M , on obtient :

$$M \neq M' \quad \text{mais} \quad \text{encrypt}(M, iv) = \text{encrypt}(M', iv)$$

Ainsi $H(M) = H(M')$: nous avons généré une collision. La fonction de hachage n'est *pas* résistante aux collisions.

b. Attaque contre l'intégrité (Encrypt-then-MAC)

Dans le paradigme *Encrypt-then-MAC*, on transmet $C = c_0 \| c_1 \| \dots$ accompagné de :

$$\text{HMAC}(K, C) = H(K \| H(K \| C))$$

La faille de H permet de forger une collision sur le message haché, ici le chiffré C .

Déroulement de l'attaque :

1. L'attaquant intercepte C et modifie les deux premiers blocs de 32 bits en inversant leur bit de poids fort, créant ainsi C' .
2. Puisque C et C' génèrent une collision dans H , $\text{HMAC}(K, C) = \text{HMAC}(K, C')$.
3. L'attaquant transmet C' avec le MAC d'origine. La vérification du MAC réussit (garantissant faussement l'intégrité), mais lors du déchiffrement de C' , la victime obtient un texte clair corrompu.

L'intégrité du système est donc **brisée**.